

Homework 02

Author: Manushri Pendekanti

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import optimize
from numpy.linalg import lstsq, cond, matrix_rank

%matplotlib inline
```

Question 1

1a

```
In [2]: rng = np.random.default_rng(43202)

train_mse_one = np.empty((200, 20 + 1))
test_mse_one = np.empty((200, 20 + 1))

X_ALL = rng.standard_normal(size=(100, 20))
X = np.column_stack((np.ones(100), X_ALL))

beta = np.zeros(20+1)

for i in range(1, 21):
    beta[i] = beta[i] = 0.4**i

for i in range(200):
    y_train = X @ beta + rng.normal(loc=0.0, scale=1.0, size=100)
    y_test = X @ beta + rng.normal(loc=0.0, scale=1.0, size=100)

y_hat = np.repeat(y_train.mean(), 100)
train_mse_one[i, 0] = np.mean((y_train - y_hat) ** 2)
test_mse_one[i, 0] = np.mean((y_test - y_hat) ** 2)

for j in range(1, 21):
    X_M = X[:, :j+1]
    b_hat = np.linalg.lstsq(X_M, y_train, rcond=None)[0]
    y_hat = X_M @ b_hat
```

```

train_mse_one[i, j] = np.mean((y_train - y_hat) ** 2)
test_mse_one[i, j] = np.mean((y_test - y_hat) ** 2)

mean_train_mse_one = train_mse_one.mean(axis=0)
mean_test_mse_one = test_mse_one.mean(axis=0)

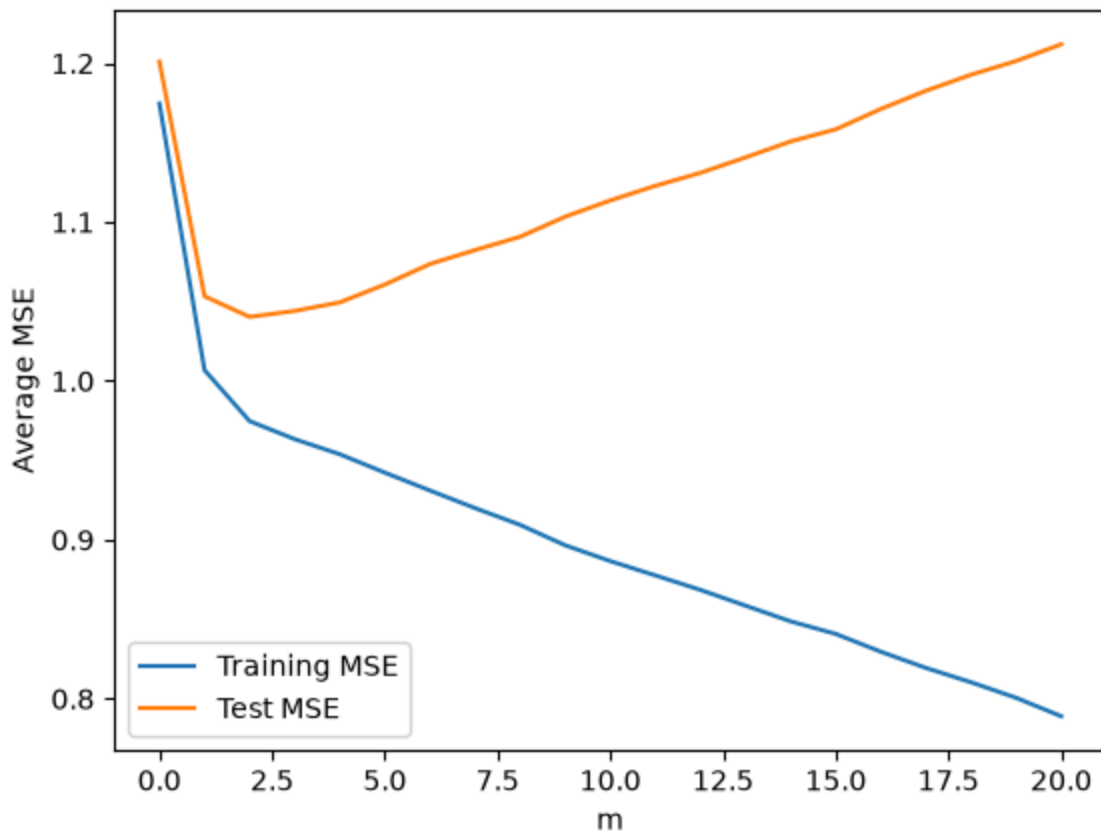
m_values = np.arange(20 + 1)

plt.plot(m_values, mean_train_mse_one, label="Training MSE")
plt.plot(m_values, mean_test_mse_one, label="Test MSE")

plt.xlabel("m")
plt.ylabel("Average MSE")
plt.legend()
plt.show()

np.all(np.diff(train_mse_one, axis=1) <= 1e-10)

```



Out[2]: np.True_

1b

```

In [3]: rng = np.random.default_rng(43202)

train_mse_one = np.empty((200, 20 + 1))
test_mse_one = np.empty((200, 20 + 1))

```

```

X_ALL = rng.standard_normal(size=(100, 20))

X = np.column_stack((np.ones(100), X_ALL))

beta = np.zeros(20+1)

for i in range(1, 21):
    beta[i] = beta[i] = 0.4**i

bias_mse_one = np.empty(21)
bias_mse_one[0] = np.mean((X @ beta - (X @ beta).mean()) ** 2)

for m in range(1, 21):
    X_m = X[:, :m + 1]
    beta_hat = np.linalg.lstsq(X_m, X @ beta, rcond=None)[0]
    y_hat = X_m @ beta_hat
    bias_mse_one[m] = np.mean((X @ beta - y_hat) ** 2)

m_values = np.arange(21)

theory_train_mse_one = bias_mse_one + \
    (1 - (m_values + 1) / 100)

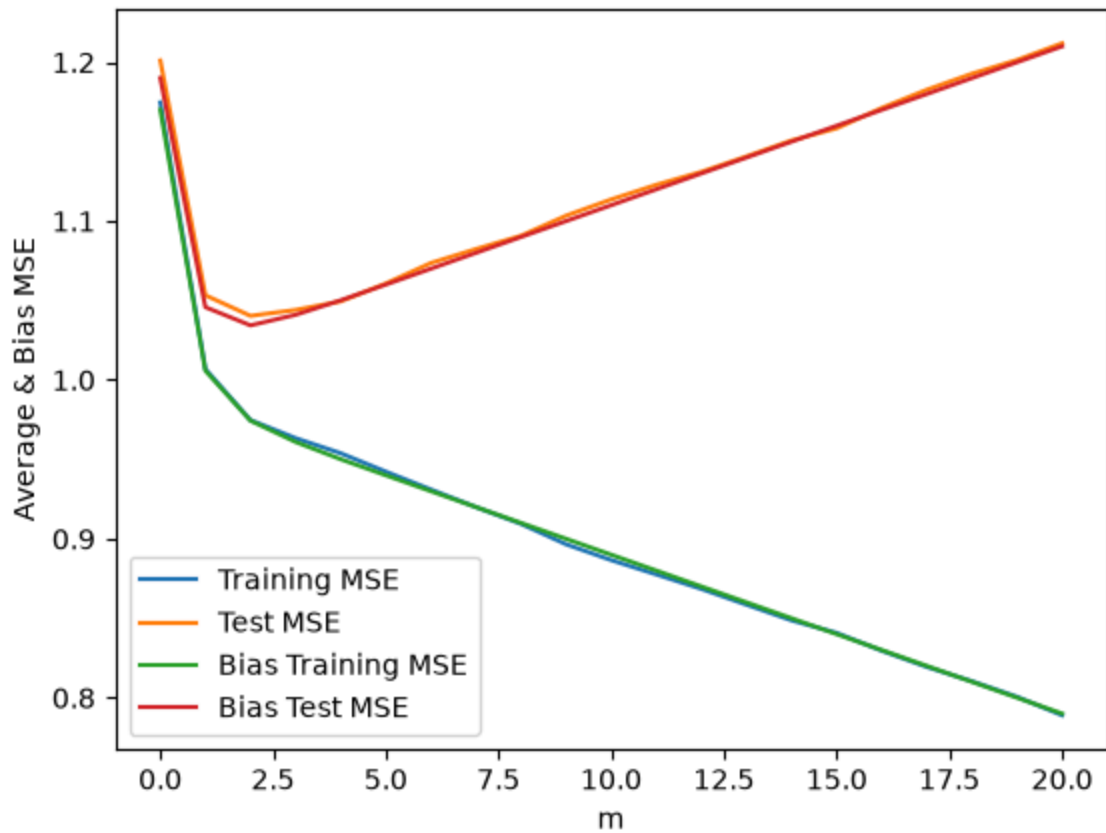
theory_test_mse_one = bias_mse_one + \
    (1 + (m_values + 1) / 100)

plt.plot(m_values, mean_train_mse_one, label="Training MSE")
plt.plot(m_values, mean_test_mse_one, label="Test MSE")

plt.plot(m_values, theory_train_mse_one, label="Bias Training MSE")
plt.plot(m_values, theory_test_mse_one, label="Bias Test MSE")

plt.xlabel("m")
plt.ylabel("Average & Bias MSE")
plt.legend()
plt.show()

```

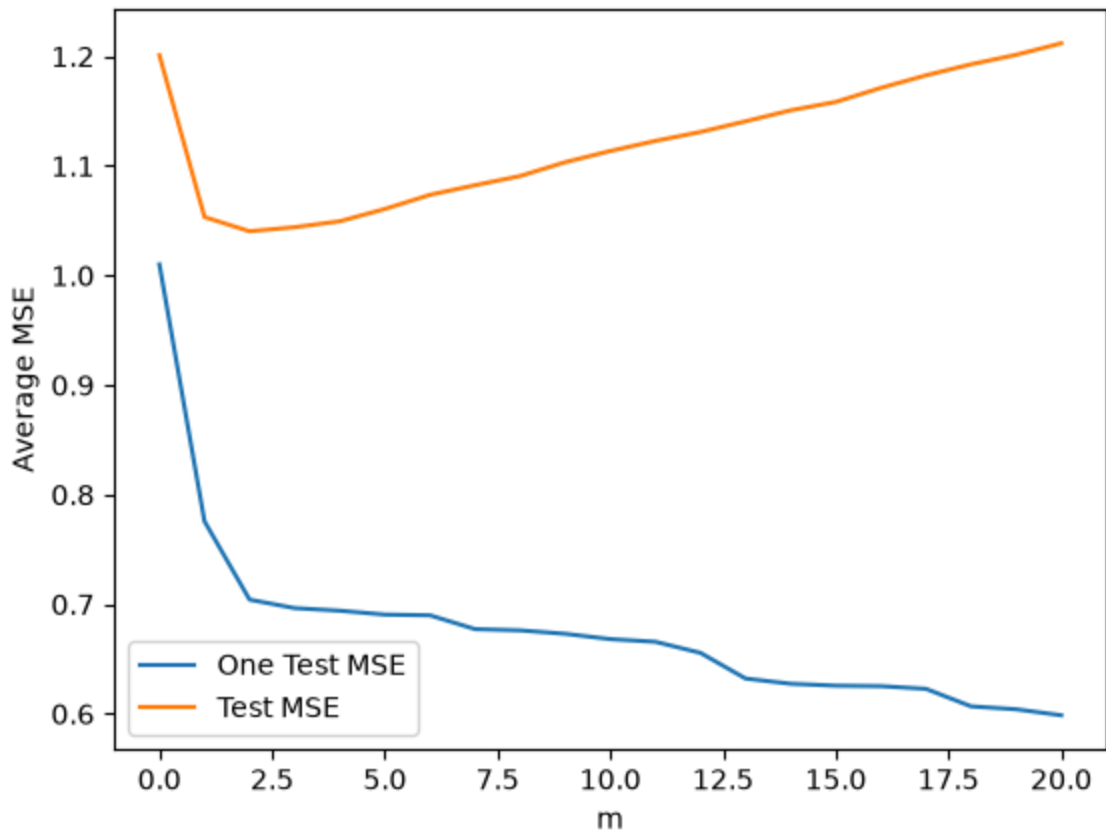


1c

```
In [4]: plt.plot(m_values, test_mse_one[0], label="One Test MSE")
plt.plot(m_values, mean_test_mse_one, label="Test MSE")

plt.xlabel("m")
plt.ylabel("Average MSE")
plt.legend()
plt.show()

#the single curve is less smooth because one test MSE has a lot more variati
#training MSE cannot be used to pick num predictors because it reduces as n
```



Question 2

2a

```
In [5]: n, p = 100, 6
i = np.arange(1, n + 1)[: , None]
j = np.arange(1, p + 1)[None, :]
X_all = np.sqrt(2) * np.cos(np.pi * j * (i - 0.5) / n)

X_0 = np.array([0,1,0,0,1,0])

B = np.array([0.5, 0.5, 0.5, 0.5, 0.5, 0.5])

u_0 = X_0.T @ B

theoretical_squared_bias = np.empty(7)

for m in range(7):
    theoretical_squared_bias[m] = 0

    for j in range(m, 6):
        theoretical_squared_bias[m] += X_0[j] * B[j]

    theoretical_squared_bias[m] = theoretical_squared_bias[m] ** 2

expected_square_error = np.empty(7)
```

```

for m in range(p + 1):
    variance_sum = 0

    for j in range(m):
        variance_sum += X_0[j] ** 2

    variance = (1 / n) * (1 + variance_sum)

    expected_square_error[m] = (
        theoretical_squared_bias[m] + variance
    )

```

#The squared bias term shows how much error comes from the predictors that are included in the model
#The variance term represents the error that comes from
#estimating the coefficients of the predictors that are included in the model

2b

```

In [6]: rng = np.random.default_rng(43203)

squared_errors = np.empty((200, p + 1))

for sim in range(200):
    y = X_all @ B + rng.normal(0, 1, size=n)
    X_m = np.ones((n, 1))
    beta_hat = np.linalg.lstsq(X_m, y, rcond=None)[0]
    u_hat = beta_hat[0]
    squared_errors[sim, 0] = (u_hat - u_0) ** 2

    for m in range(1, p + 1):
        X_m = np.column_stack(
            (np.ones(n), X_all[:, :m])
        )
        beta_hat = np.linalg.lstsq(
            X_m, y, rcond=None
        )[0]
        u_hat = beta_hat[0]

        for j in range(m):
            u_hat += X_0[j] * beta_hat[j + 1]

        squared_errors[sim, m] = (
            u_hat - u_0
        ) ** 2

```

```

simulation_average = squared_errors.mean(axis=0)

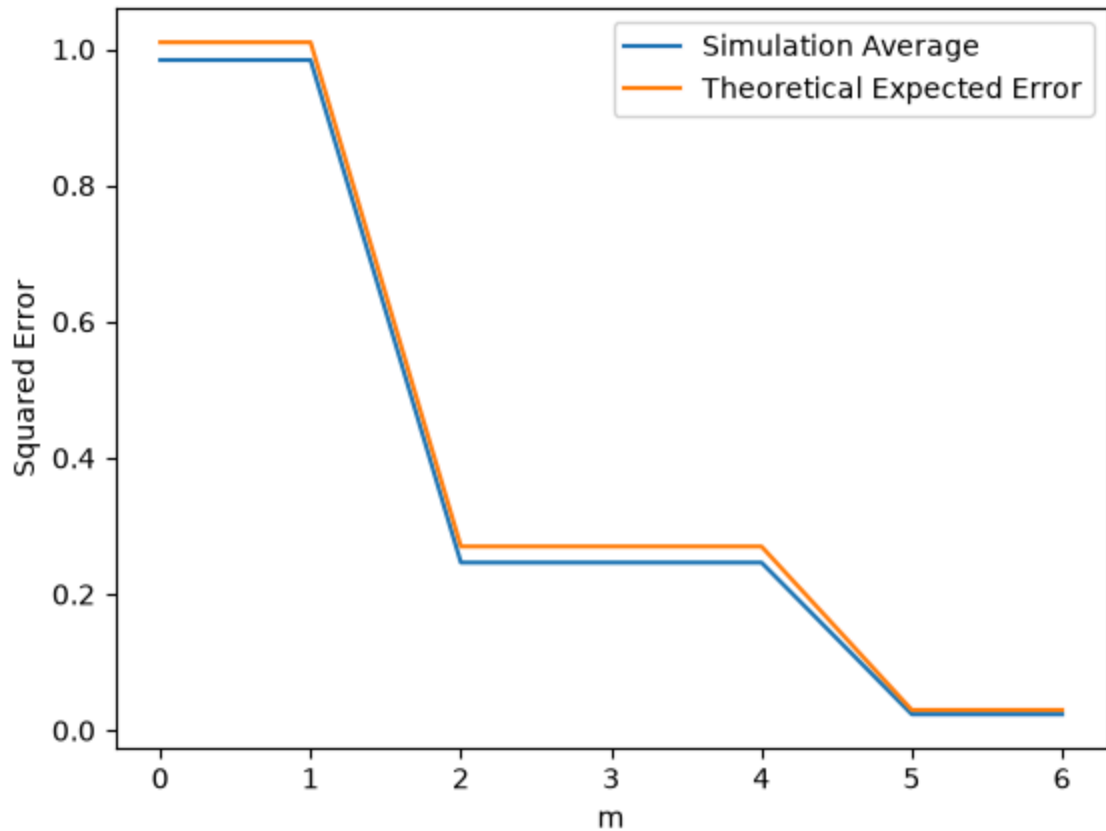
m_values = np.arange(p + 1)

plt.plot(
    m_values,
    simulation_average,
    label="Simulation Average"
)

plt.plot(
    m_values,
    expected_square_error,
    label="Theoretical Expected Error"
)

plt.xlabel("m")
plt.ylabel("Squared Error")
plt.legend()
plt.show()

```



2c

In [7]: *#only 2 contribute directly to mean squared response because other variables
#not zero are only adding to the error and not providing meaningful informat
#way to predict the respoinse and the target error dipped sharply between or
#but stayed constant at the other few but average decreased more gradually*

Question 3

3a

$$\begin{aligned}
 3a.) \text{Mse}_{\text{train}} &= \frac{1}{n} \|y - Hy\|_2^2 \\
 &= y - Hy = (I - H)y \\
 &= (I - H)\mu + (I - H)\epsilon \\
 &= \frac{1}{n} \|(I - H)\mu + (I - H)\epsilon\|^2 \\
 &= \frac{1}{n} [\|(I - H)\mu\|^2 + 2\mu^T(I - H)\epsilon + \epsilon^T(I - H)\epsilon] \\
 &= \frac{1}{n} \underbrace{\|(I - H)\mu\|^2}_{= B^2} + \underbrace{2\mu^T(I - H)\epsilon}_0 + \underbrace{\epsilon^T(I - H)\epsilon}_{\sigma^2 \text{tr}(I - H)} \\
 & \qquad \qquad \qquad = n - (p+1) \\
 &= \frac{1}{n} [B^2 + \sigma^2(n - (p+1))]
 \end{aligned}$$

$$\begin{aligned}
 \text{Mse}_{\text{test}} &= \frac{1}{n} \|y^* - Hy\|^2 \\
 y^* &= \mu + \epsilon^* \\
 y^* - Hy &= \mu + \epsilon^* - H(\mu + \epsilon) \\
 &= \mu + \epsilon^* - H\mu - H\epsilon \\
 &= (I - H)\mu + \epsilon^* - H\epsilon \\
 &= \frac{1}{n} \|(I - H)\mu + \epsilon^* - H\epsilon\|^2 \\
 &= \frac{1}{n} \underbrace{\|(I - H)\mu\|^2}_{B^2} + \underbrace{2\mu^T(I - H)\epsilon^*}_{n\sigma^2} + \underbrace{\epsilon^{*T}(I - H)\epsilon}_{\sigma^2(p+1)} \\
 &= \frac{1}{n} [B^2 + n\sigma^2 + \sigma^2(p+1)]
 \end{aligned}$$

3b

$$\begin{aligned}
 3b.) \mathbb{E}[\text{MSE}_{\text{test}}|X] &= \frac{B^2}{n} + \sigma^2 \left(1 + \frac{p+1}{n}\right) - \frac{B^2}{n} + \sigma^2 \left(1 - \frac{p+1}{n}\right) \\
 &= 0.04 + 1 \left(1 + \frac{5}{120}\right) - \left(0.04 + 1 \left(1 - \frac{5}{120}\right)\right) \\
 &= \boxed{1.08167} - \boxed{0.99833} \\
 &= \boxed{0.08333}
 \end{aligned}$$

In [8]: `#the ordering need not hold because at the end of the day each of test and t`
`#that was not predicted before resulting the error changes`

3c

$$3c.) \text{Mse test} = \frac{116.4 + 2(5)(0.96)}{120} = \boxed{1.05}$$

$$cp = \frac{116.4}{0.96} - 120 + 2(5) = \boxed{11.25}$$

$$\begin{aligned} \text{Related: } \frac{RSS}{\sigma^2} - n + 2(p+1) &= cp \\ &= \frac{RSS}{\sigma^2} + 2(p+1) \rightarrow \text{added } n \\ &= \frac{RSS + 2(p+1)\sigma^2}{\sigma^2} \rightarrow \text{multiplied } \sigma^2 \\ \text{Mse test} &= \frac{RSS + 2(p+1)\sigma^2}{n} \rightarrow \text{divide by } n \end{aligned}$$

Question 4

4a

```
In [9]: df = pd.read_csv("diabetes.csv")[0:371]

y = df["y"]

#Model A

X_a = df[['bmi', 'bp', 's5', 'sex', 's1', 's2', 's4']]

B_hat_a = np.linalg.lstsq(X_a, y, rcond=None)[0]

p_a = 7

df_a = 370 - 8

y_hat_a = X_a @ B_hat_a

RSS_a = np.sum((y - y_hat_a)**2)

sigma_a = RSS_a / df_a

#Model b

X_b = df[['bmi', 'bp', 's5', 'sex', 's1', 's2', 's4', 's6']]

B_hat_b = np.linalg.lstsq(X_b, y, rcond=None)[0]

p_b = 8

df_b = 370 - 9

y_hat_b = X_b @ B_hat_b

RSS_b = np.sum((y - y_hat_b)**2)
```

```

sigma_b = RSS_b / df_b

X_full = df.drop(columns = "y")

B_hat_full = np.linalg.lstsq(X_full, y, rcond= None)[0]

p_full = 10

df_full = 370 - 11

y_hat_full = X_full @ B_hat_full

RSS_full = np.sum((y - y_hat_full)**2)

sigma_full = RSS_full / df

```

4b

```

In [10]: n = 370

# Model A
Cp_a = RSS_a / sigma_a - n + 2 * (p_a + 1)
AIC_a = n * np.log(RSS_a / n) + 2 * (p_a + 1)
BIC_a = n * np.log(RSS_a / n) + (p_a + 1) * np.log(n)

# Model B
Cp_b = RSS_b / sigma_b - n + 2 * (p_b + 1)
AIC_b = n * np.log(RSS_b / n) + 2 * (p_b + 1)
BIC_b = n * np.log(RSS_b / n) + (p_b + 1) * np.log(n)

print("Model A:", Cp_a, AIC_a, BIC_a)
print("Model B:", Cp_b, AIC_b, BIC_b)

#All criterons would selected Model A because model A has smaller values in

```

Model A: 8.0 3782.3664714706597 3813.674495515766

Model B: 9.0 3783.9073331734903 3819.128860224235

4c

```

In [11]: # the while there is an improvement of fit or decreased RSS but the accordin
#criteria model A is the best fit and just because we selected a model it do
#not showcase how the data was selected because this was a selection basedx
# on its performance on a set of criteria

```

Question 5

5a

```
In [12]: #the first step that misuses the final test data is step e because it decides
#to pick it model based on the same data that it going to do testing on and
#the smallest one and the reason picking smallest one is too favorable
#becuase if we are selecting a smallest error based on the same
#dataset that will eventually perform testing one obviously it will
#perform better because we picked it based on that performance
```

5b

```
In [13]: #I would perform five-fold cross-validation using only the 370 training obse
#for each of the eleven candidate models. I would select the predictors with
# I would then refit that selected model using all 370 training observations
#Finally, I would evaluate this model once on rows
#371-442 and report its test MSE as the final estimate of prediction error
```

5c

```
In [14]: # If the analyst has already evaluated the 11 test MSEs, they can report whi
# had the smallest MSE, but they should mention that the test data was used
# They would need a get a new set of test data to evaluate the selected mode
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```